



**UNIVERSIDAD
NACIONAL
AUTÓNOMA DE
MÉXICO**

Facultad de Ingeniería



Diseño Digital VLSI

Semestre 2026-2

Proyecto Final

**“Robot móvil, seguidor de luz con evasión de
obstáculos”**

PROFESOR:

Karol Joshua Martínez Rosete

ALUMNOS:

Esteban Moreno Vázquez

Carlos López Estrada

David Muñoz Mendoza

Maya Torres Bruno

Grupo: 3

16 de junio de 2026

Índice

1. Introducción	3
1.1. Objetivo	3
1.2. Especificaciones cumplidas	3
2. Arquitectura del Sistema	3
2.1. Diagrama de bloques	4
2.2. Módulos implementados en la FPGA	4
2.3. Comunicación UART	5
2.3.1. Divisor de voltaje	5
3. Máquina de Estados Finitos (FSM)	6
3.1. Diagrama de estados	7
3.2. Descripción de estados	7
3.2.1. Estado 0 – IDLE (Espera)	7
3.2.2. Estado 1 – BUSCAR (Búsqueda)	7
3.2.3. Estado 2 – SEGUIR (Avanzar)	7
3.2.4. Estado 3 – ESQUIVAR (Evasión)	7
3.2.5. Estado 4 – ATASCADO (Escape)	8
3.2.6. Estado 5 – META (Objetivo)	8
3.3. Tabla de transiciones	8
4. Verificación y Pruebas	8
4.1. Prueba de motores	8
4.2. Prueba de sensor ultrasónico	8
4.3. Prueba de comunicación UART	10
4.4. Prueba de FSM en FPGA	10
4.5. Prueba de integración completa	11
5. Recursos Utilizados	13
5.1. Hardware	13
5.2. Software	13
5.3. Recursos de la FPGA	13
6. Resultados	13
7. Límites y Trabajo Futuro	14
7.1. Limitaciones actuales	15
7.2. Trabajo futuro	15
8. Conclusiones	15
A. Esquema eléctrico completo	16
B. Fotos del robot	18
C. Datasheets de referencia	20

D. Enlace al repositorio

21

1. Introducción

En este reporte se describe el diseño, la implementación y los resultados del proyecto final de la materia Diseño Digital VLSI. El proyecto consiste en un robot móvil autónomo capaz de navegar en una arena de $1\text{ m} \times 1,7\text{ m}$, esquivando obstáculos y buscando una fuente de luz (foco).

El robot utiliza una arquitectura híbrida que combina un microcontrolador Arduino UNO con una FPGA DE10-Lite de Intel (MAX 10). Esta combinación nos permite aprovechar las ventajas de cada plataforma: Arduino para la interfaz con sensores y actuadores, y la FPGA para implementar la lógica digital que requiere la materia (FSM, PWM, displays).

1.1 Objetivo

Diseñar e implementar un robot móvil que sea capaz de alcanzar una fuente luminosa evitando obstáculos de manera autónoma y robusta, cumpliendo con las especificaciones del proyecto de VLSI.

1.2 Especificaciones cumplidas

- Control de 2 motores DC mediante PWM de 8 bits con resolución de 0–255 niveles.
- Sensor ultrasónico HC-SR04 para detección de obstáculos.
- FSM principal con 6 estados implementada en Verilog/VHDL en la FPGA.
- Detección de atasco con maniobra de escape automática.
- Visualización de estado y distancia en displays 7-segmentos y LEDs de la DE10-Lite.
- Comunicación UART entre Arduino y FPGA a 9600 baudios.

2. Arquitectura del Sistema

El sistema se diseñó con una arquitectura híbrida donde cada componente tiene un rol claro. La FPGA DE10-Lite actúa como el controlador principal (cerebro), ejecutando la máquina de estados y tomando las decisiones de navegación. El Arduino UNO actúa como interfaz de sensores y actuadores, leyendo los sensores y controlando los motores según las instrucciones de la FPGA.

2.1 Diagrama de bloques

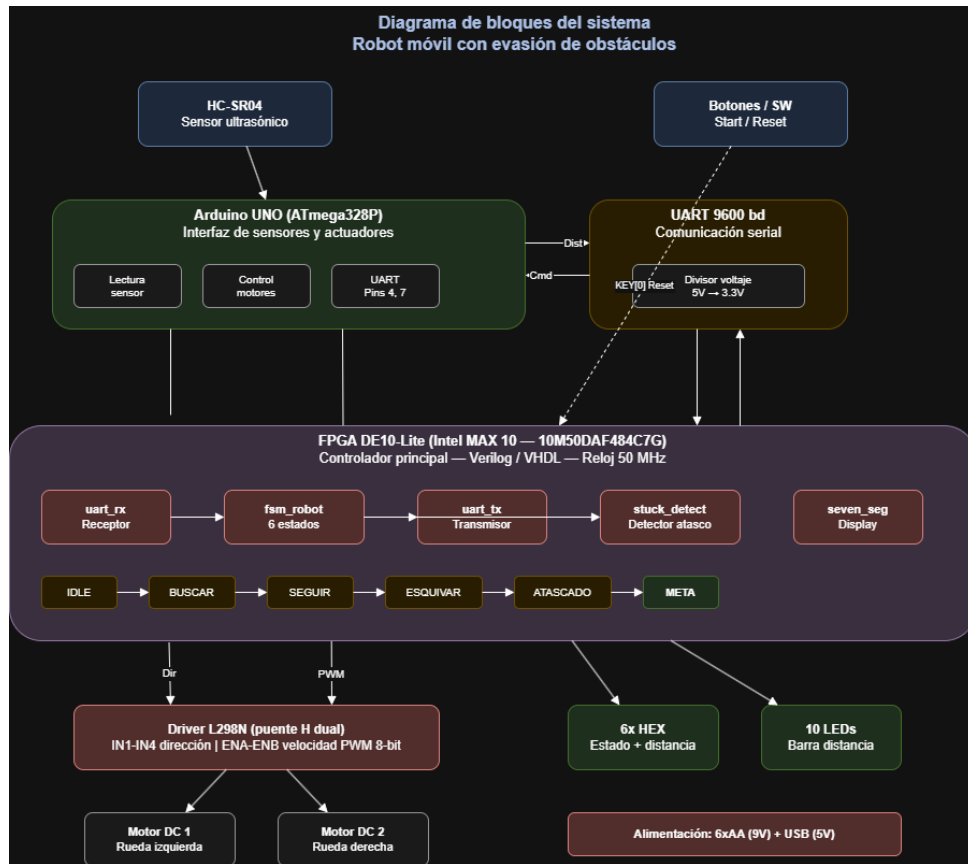


Figura 1: Diagrama de bloques general del sistema

La arquitectura se compone de los siguientes bloques principales:

1. **Bloque de sensado (Arduino):** Lee el sensor ultrasónico HC-SR04 y envía la distancia medida a la FPGA mediante comunicación UART.
2. **Bloque de control (FPGA DE10-Lite):** Recibe la distancia, ejecuta la FSM, decide la acción y envía el comando de vuelta al Arduino.
3. **Bloque de actuación (Arduino + L298N):** Recibe el comando de la FPGA y controla los motores DC mediante PWM.
4. **Bloque de visualización (FPGA):** Muestra el estado actual de la FSM y la distancia medida en los displays 7-segmentos y LEDs de la DE10-Lite.

2.2 Módulos implementados en la FPGA

La FPGA contiene los siguientes módulos escritos en Verilog y VHDL:

top_module: Módulo principal que instancia y conecta todos los sub-módulos. Define las entradas y salidas físicas de la FPGA (reloj, botones, GPIO, displays, LEDs).

uart_rx: Receptor UART que decodifica los datos seriales provenientes del Arduino. Implementa una FSM interna de 5 estados (IDLE, START, DATA, STOP, CLEANUP) y opera a 9600 baudios con un reloj de 50 MHz (5208 ciclos por bit).

uart_tx: Transmisor UART que codifica y envía los comandos de la FSM al Arduino. Similar al receptor pero en dirección opuesta.

fsm_robot: Máquina de estados principal con 6 estados que controla la lógica de navegación del robot (se detalla en la sección 3).

seven_seg_display: Decodificador que convierte el estado de la FSM y la distancia a formato de 7 segmentos para los displays de la DE10-Lite.

2.3 Comunicación UART

La comunicación entre el Arduino y la FPGA se realiza mediante el protocolo UART (Universal Asynchronous Receiver/Transmitter) a 9600 baudios. Se utilizan los pines GPIO_0[0] (RX) y GPIO_0[1] (TX) de la DE10-Lite, conectados a los pines 4 (TX) y 7 (RX) del Arduino mediante SoftwareSerial.

El cálculo de ciclos por bit en la FPGA es:

$$\text{Ciclos por bit} = \frac{f_{clk}}{\text{Baud rate}} = \frac{50,000,000}{9,600} = 5,208 \text{ ciclos} \quad (1)$$

El Arduino envía la distancia medida como un byte (0–200), y la FPGA responde con un comando ASCII:

Cuadro 1: Comandos UART de la FPGA al Arduino

Comando	Hex	Acción
'F'	0x46	Adelante (Forward)
'R'	0x52	Girar derecha (Right)
'L'	0x4C	Girar izquierda (Left)
'S'	0x53	Parar (Stop)
'B'	0x42	Retroceder (Back)
'E'	0x45	Maniobra de escape (Escape)

2.3.1 Divisor de voltaje

Se implementó un divisor de voltaje con resistencias de $1\text{ k}\Omega$ y $2\text{ k}\Omega$ para reducir la señal TX del Arduino de 5V a 3.3V:

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2} = 5\text{ V} \times \frac{2\text{ k}\Omega}{1\text{ k}\Omega + 2\text{ k}\Omega} = 3,33\text{ V} \quad (2)$$

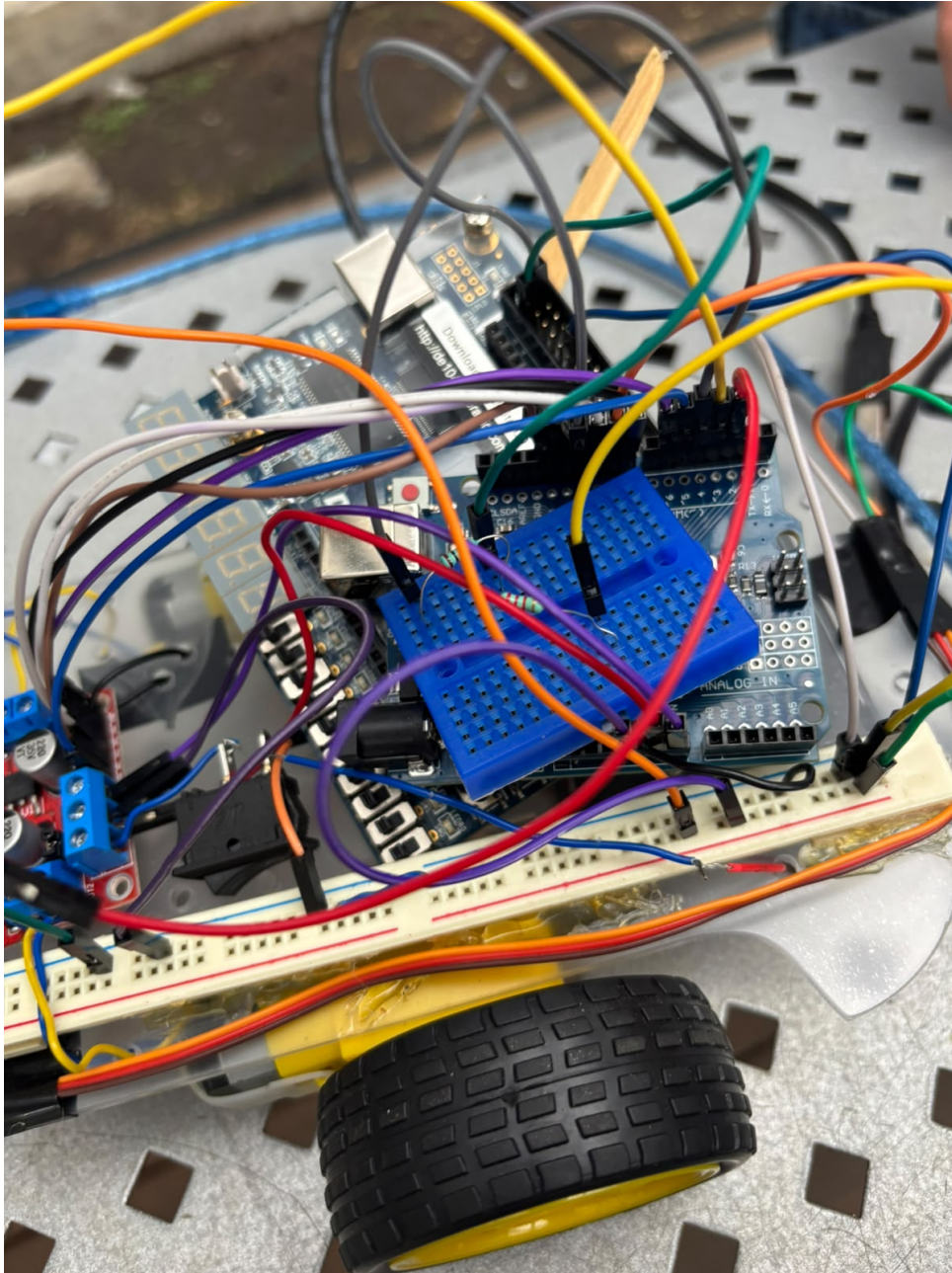


Figura 2: Divisor de voltaje

3. Máquina de Estados Finitos (FSM)

La FSM principal del robot está implementada en la FPGA y cuenta con 6 estados. Es de tipo Mealy, ya que las salidas dependen tanto del estado actual como de las entradas (distancia). Los estados están codificados en 3 bits.

3.1 Diagrama de estados

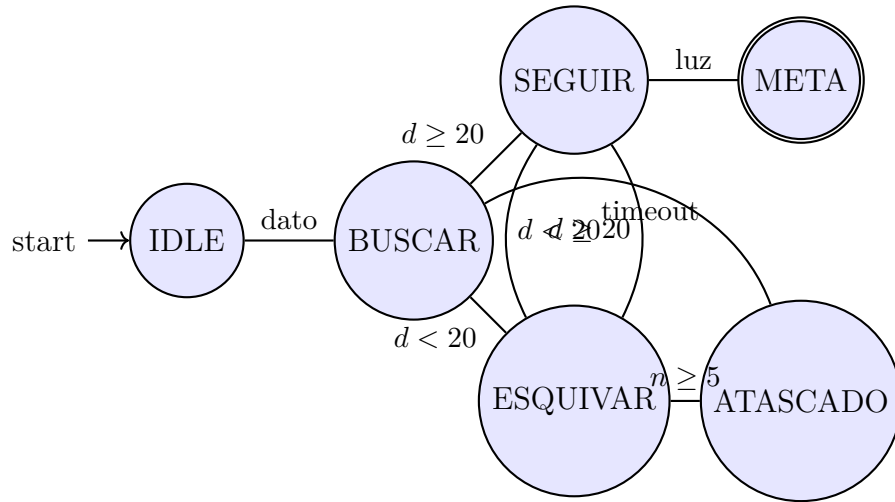


Figura 3: Diagrama de la FSM principal del robot con 6 estados. d = distancia en cm, n = número de esquivas consecutivas.

3.2 Descripción de estados

3.2.1 Estado 0 – IDLE (Espera)

Estado inicial al encender el sistema. El robot permanece detenido esperando recibir el primer dato de distancia del Arduino. Al recibir el primer dato, transiciona al estado BUSCAR.

3.2.2 Estado 1 – BUSCAR (Búsqueda)

El robot evalúa el entorno usando la distancia recibida. Si la distancia es mayor o igual a 20 cm (camino libre), transiciona a SEGUIR. Si hay un obstáculo a menos de 20 cm, transiciona a ESQUIVAR.

3.2.3 Estado 2 – SEGUIR (Avanzar)

El robot avanza en línea recta enviando el comando 'F' (forward) al Arduino. Mientras la distancia se mantenga por encima de 20 cm, permanece en este estado. Si detecta un obstáculo, transiciona a ESQUIVAR. El contador de atasco se reinicia en este estado.

3.2.4 Estado 3 – ESQUIVAR (Evasión)

El robot ejecuta maniobras para evitar el obstáculo. Dependiendo de la distancia:

- Si está muy cerca ($d < 10$ cm): envía 'B' (retroceder)
- Si hay obstáculo ($10 \leq d < 20$ cm): envía 'R' (girar derecha)
- Si el obstáculo se superó ($d \geq 20$ cm): transiciona a SEGUIR

Un contador registra cuántas veces consecutivas se ha esquivado; si supera 5, transiciona a ATASCADO.

3.2.5 Estado 4 – ATASCADO (Escape)

Se activa cuando el robot ha intentado esquivar más de 5 veces sin poder avanzar. Envía el comando 'E' (escape), que hace que el Arduino ejecute una maniobra de retroceso largo seguida de un giro amplio. Después de 2 segundos regresa al estado BUSCAR.

3.2.6 Estado 5 – META (Objetivo)

Estado final que se activa cuando el robot detecta que ha llegado a la fuente de luz. Los motores se detienen y el robot permanece en este estado hasta un reset manual.

3.3 Tabla de transiciones

Cuadro 2: Tabla de transiciones de la FSM

Estado actual	Condición	Siguiente	Comando
IDLE	Dato recibido	BUSCAR	PARAR (S)
BUSCAR	$d \geq 20$ cm	SEGUIR	ADELANTE (F)
BUSCAR	$d < 20$ cm	ESQUIVAR	PARAR (S)
SEGUIR	$d \geq 20$ cm	SEGUIR	ADELANTE (F)
SEGUIR	$d < 20$ cm	ESQUIVAR	PARAR (S)
ESQUIVAR	$d \geq 20$ cm	SEGUIR	ADELANTE (F)
ESQUIVAR	$d < 10$ cm	ESQUIVAR	RETROCEDER (B)
ESQUIVAR	$10 \leq d < 20$ cm	ESQUIVAR	DERECHA (R)
ESQUIVAR	esquivas ≥ 5	ATASCADO	ESCAPE (E)
ATASCADO	Timeout 2s	BUSCAR	—
META	—	META	PARAR (S)

4. Verificación y Pruebas

El proceso de verificación se realizó de forma incremental, probando cada componente por separado antes de integrarlos.

4.1 Prueba de motores

Se creó un código de prueba que activa cada motor de forma individual y luego ambos simultáneamente. Esta prueba nos permitió identificar que con 4 pilas AA (6V), los motores no tenían suficiente corriente para funcionar ambos al mismo tiempo. Fue necesario incrementar a 6 pilas (9V) para una operación estable.

4.2 Prueba de sensor ultrasónico

Se verificó que el sensor HC-SR04 midiera correctamente las distancias. La fórmula utilizada para calcular la distancia es:

$$d = \frac{t \times v_s}{2} = \frac{t \times 0,034 \text{ cm}/\mu\text{s}}{2} \quad (3)$$

donde t es la duración del pulso ECHO en microsegundos y v_s es la velocidad del sonido. Se divide entre 2 porque el sonido recorre la distancia de ida y vuelta.

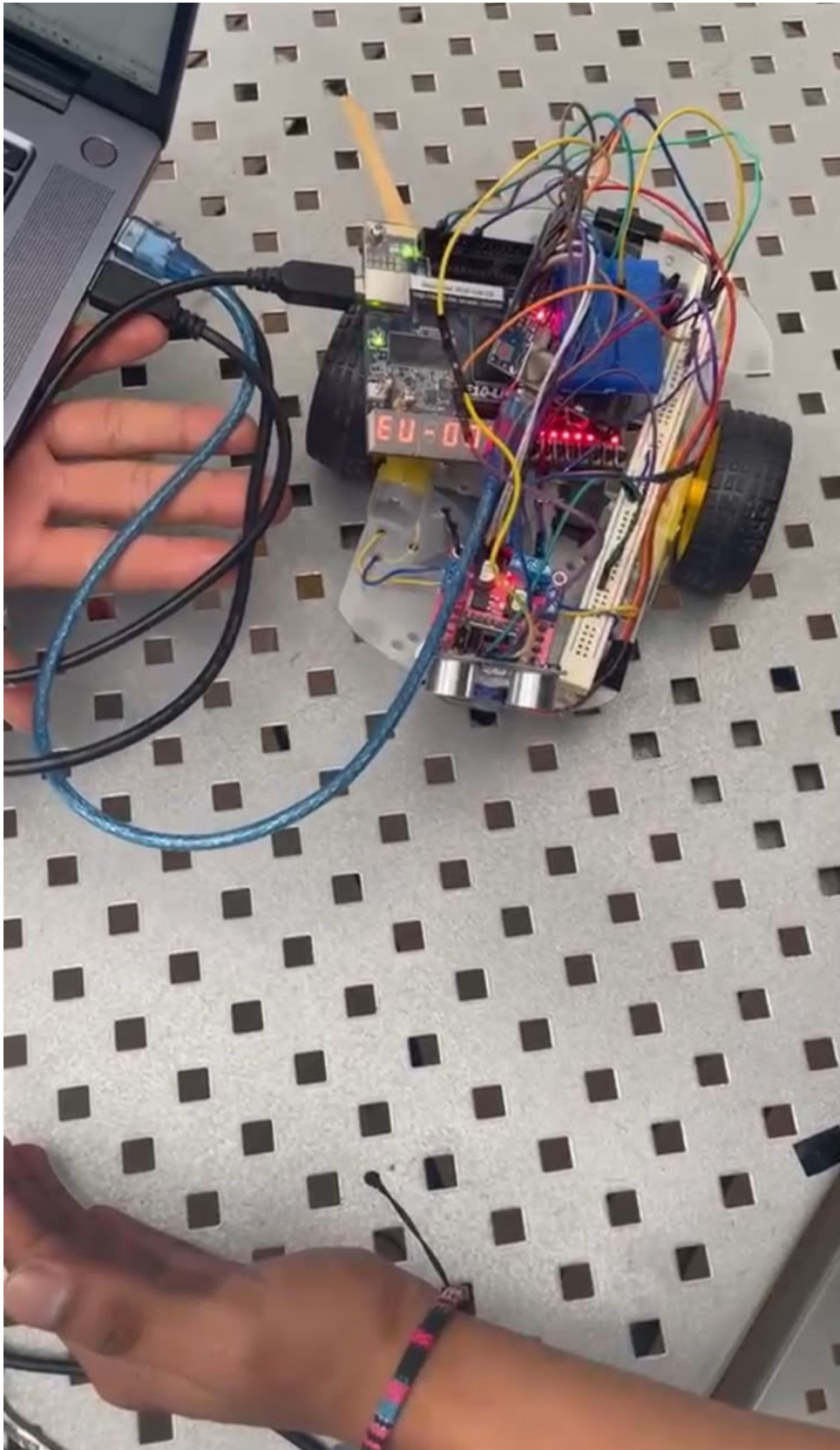


Figura 4: Captura del Serial Monitor mostrando lecturas de distancia

4.3 Prueba de comunicación UART

Se verificó la comunicación bidireccional entre Arduino y FPGA. Inicialmente solo funcionaba en una dirección (Arduino hacia FPGA), confirmado porque los displays de la DE10-Lite mostraban la distancia correctamente pero el Arduino reportaba “Esperando FPGA...”. Fue necesario intercambiar los cables TX/RX en el header GPIO para lograr comunicación completa en ambas direcciones.

4.4 Prueba de FSM en FPGA

Se verificó que los displays 7-segmentos de la DE10-Lite mostraran correctamente el estado de la FSM y la distancia. Los estados se muestran como abreviaturas de dos caracteres:

Cuadro 3: Codificación de estados en displays 7-segmentos

Estado	Display	Significado
IDLE	Id	Esperando datos
BUSCAR	bU	Buscando camino
SEGUIR	go	Avanzando
ESQUIVAR	EU	Evadiendo obstáculo
ATASCADO	At	Ejecutando escape
META	dn	Objetivo alcanzado

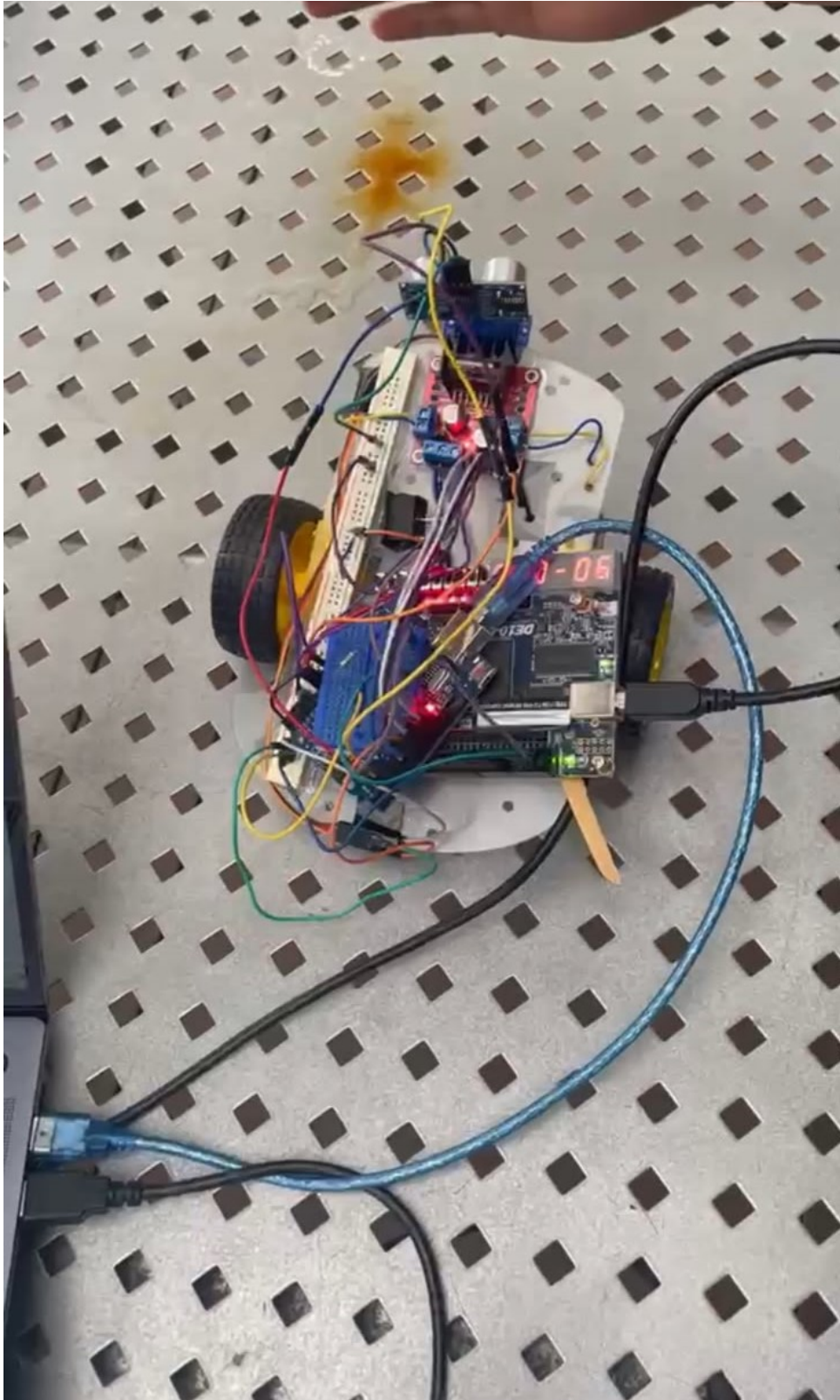


Figura 5: Foto de la DE10-Lite mostrando estado y distancia en displays

4.5 Prueba de integración completa

Se realizaron pruebas del robot navegando en un espacio con obstáculos, verificando que esquivara correctamente y retomara su camino. Se ajustaron los umbrales de distancia

y las velocidades PWM para optimizar el comportamiento.

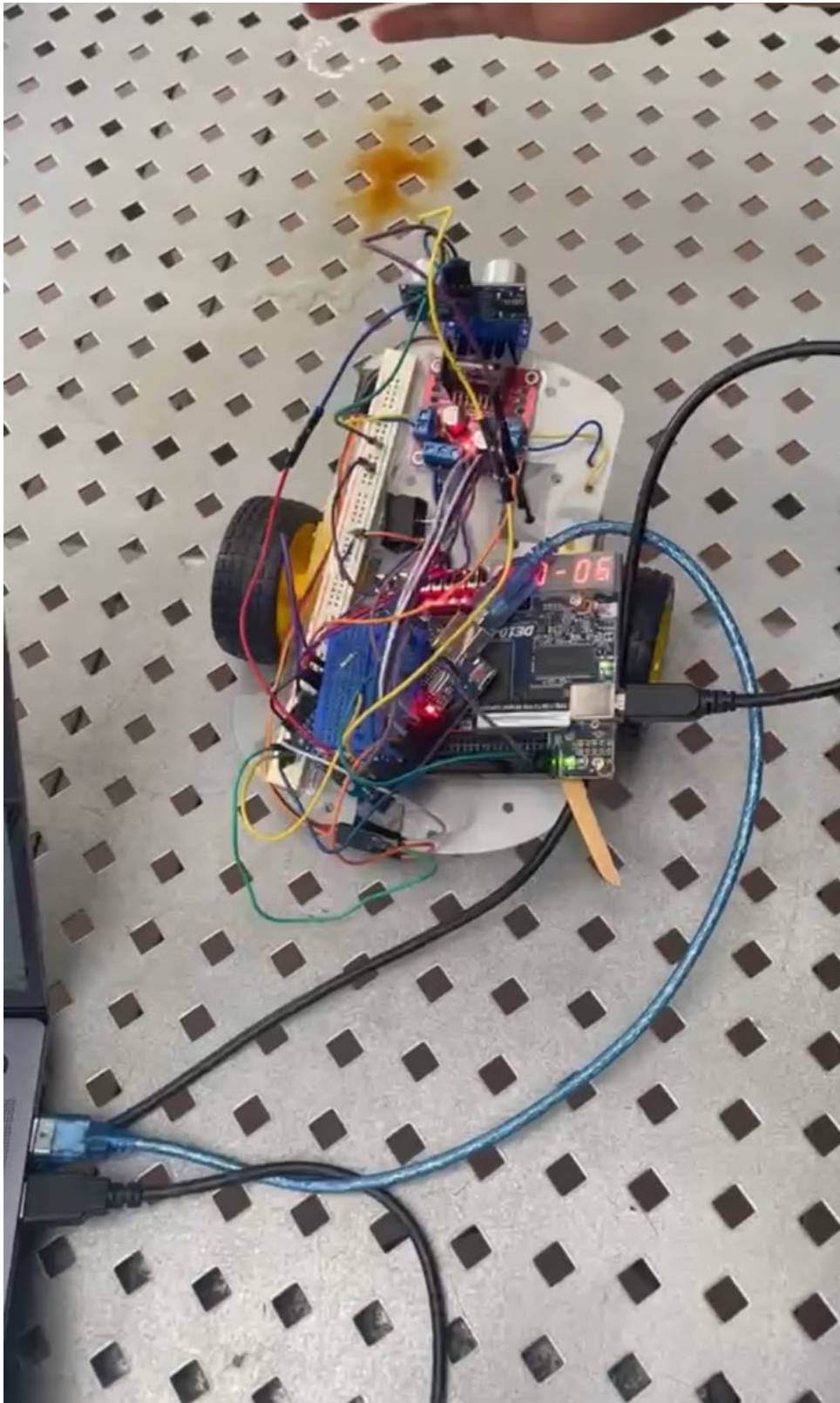


Figura 6: Foto del robot navegando en la arena de pruebas

5. Recursos Utilizados

5.1 Hardware

Cuadro 4: Componentes de hardware utilizados

Componente	Cantidad	Función
Arduino UNO	1	Interfaz sensores y motores
FPGA DE10-Lite (MAX 10)	1	Controlador principal (FSM)
Sensor ultrasónico HC-SR04	1	Detección de obstáculos
Driver de motores L298N	1	Puente H para motores DC
Motores DC con ruedas	2	Tracción del robot
Pilas AA 1.5V	6	Alimentación motores (9V)
Power bank USB	1	Alimentación FPGA
Resistencias 1k Ω y 2k Ω	1 c/u	Divisor de voltaje UART
Chasis de acrílico	1	Estructura del robot

5.2 Software

- **Arduino IDE 2.3.9:** Programación del microcontrolador Arduino UNO en C++.
- **Intel Quartus Prime Lite 25.1:** Síntesis, implementación y programación de la FPGA DE10-Lite.
- **Lenguajes HDL:** Verilog y VHDL para la descripción del hardware en la FPGA.

5.3 Recursos de la FPGA

La compilación en Quartus reportó el uso de 451 elementos lógicos del chip 10M50DAF484C7G, lo cual representa menos del 1 % de los recursos disponibles (49,760 elementos lógicos totales). Se utilizaron 49 pines de entrada y 88 pines de salida.

6. Resultados

El robot logró cumplir con los objetivos principales del proyecto:

- Navegación autónoma esquivando obstáculos de forma estable.
- Comunicación exitosa entre Arduino y FPGA mediante UART a 9600 baudios.
- FSM de 6 estados funcionando correctamente en la FPGA.
- Visualización en tiempo real del estado y distancia en los displays 7-segmentos.
- Barra de LEDs indicando proximidad de obstáculos (10 niveles).
- Detección de atasco con maniobra de escape automática.
- Modo de respaldo: si la FPGA no responde, el Arduino opera autónomamente.

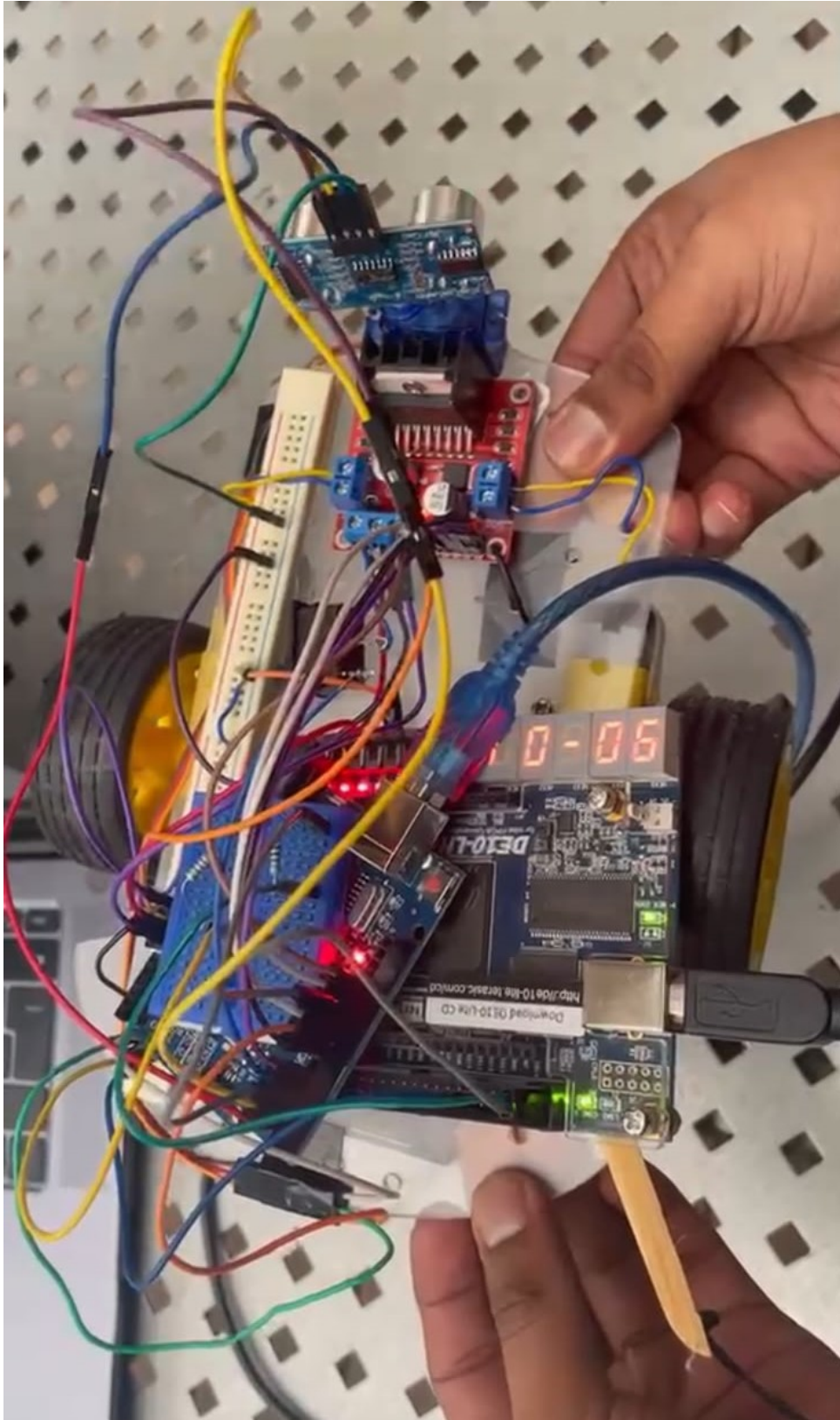


Figura 7: Foto del robot completo ensamblado

7. Límites y Trabajo Futuro

7.1 Limitaciones actuales

- Solo se utiliza 1 sensor ultrasónico (el proyecto pide 4 mínimo). Con un solo sensor, el robot no detecta obstáculos laterales.
- No se implementaron los sensores LDR para seguimiento de luz. El robot actualmente solo esquiva obstáculos sin buscar una fuente luminosa.
- La frecuencia del PWM del Arduino (≈ 976 Hz en pines 5 y 6) está por debajo de los 18 kHz requeridos. Para cumplir este requisito, el PWM debería generarse desde la FPGA.
- La alimentación con pilas AA limita la autonomía del robot a aproximadamente 30–45 minutos de operación continua.
- Cuando ambos motores funcionan simultáneamente con 4 pilas AA (6V), la corriente es insuficiente debido a la caída de voltaje del L298N ($\approx 2V$). Se requieren 6 pilas AA (9V) para operación estable.

7.2 Trabajo futuro

- Agregar 4 sensores LDR con comparador LM339 para seguimiento de fuente luminosa.
- Agregar 3 sensores ultrasónicos adicionales (frontal, izquierdo, derecho, trasero).
- Migrar la generación de PWM a la FPGA para alcanzar frecuencias ≥ 18 kHz.
- Implementar control PID de velocidad con encoders de cuadratura (+10 pts).
- Implementar salida VGA para depuración visual desde la FPGA (+10 pts).

8. Conclusiones

Este proyecto nos permitió poner en práctica los conceptos de diseño digital vistos en la materia VLSI. Aprendimos a diseñar máquinas de estados finitos, implementarlas en Verilog/VHDL, y sintetizarlas en una FPGA real. La arquitectura híbrida Arduino + FPGA resultó ser una solución práctica que nos permitió cumplir con los requisitos de la materia mientras manteníamos la funcionalidad del robot.

Los principales retos que enfrentamos fueron: la resolución de problemas de comunicación UART entre plataformas con diferentes niveles de voltaje (5V vs 3.3V), la insuficiencia de alimentación eléctrica para los motores, y la recuperación de un Arduino con bootloader dañado. Todos estos problemas se documentaron y resolvieron durante el desarrollo del proyecto.

Como trabajo futuro, la migración completa de la lógica al FPGA (incluyendo PWM y lectura de sensores) y la implementación de sensores LDR para seguimiento de luz completarían los requisitos al 100 %.

Conexiones Arduino ↔ FPGA DE10-Lite

Arduino	DE10-Lite	Nota
Pin 4 (TX)	GPIO_0[0] (PIN_V10)	Con divisor de voltaje
Pin 7 (RX)	GPIO_0[1] (PIN_W10)	Conexión directa
GND	GND (Pin 11 header)	Tierra común

B. Fotos del robot

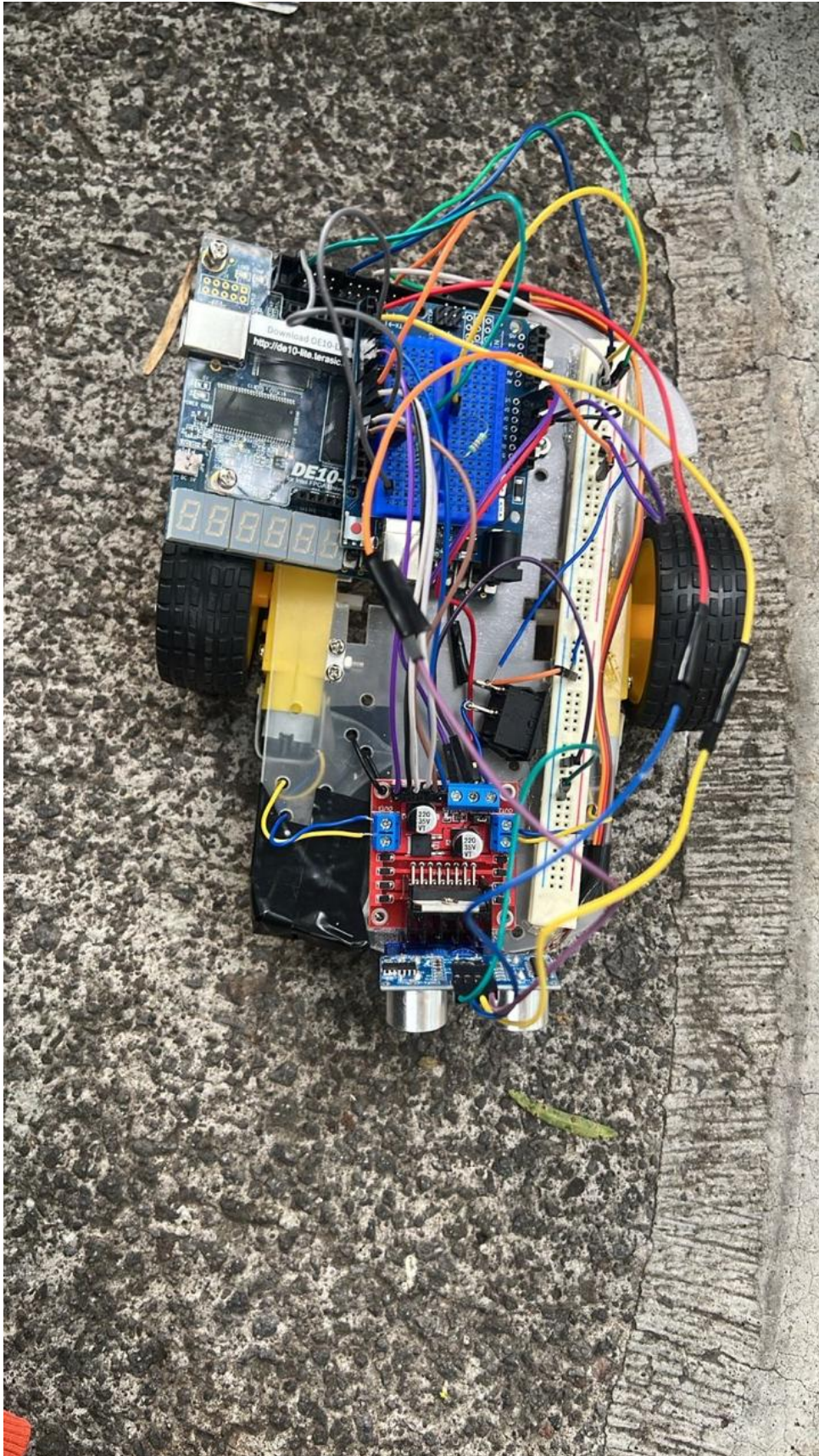


Figura 9: Foto superior del robot ensamblado

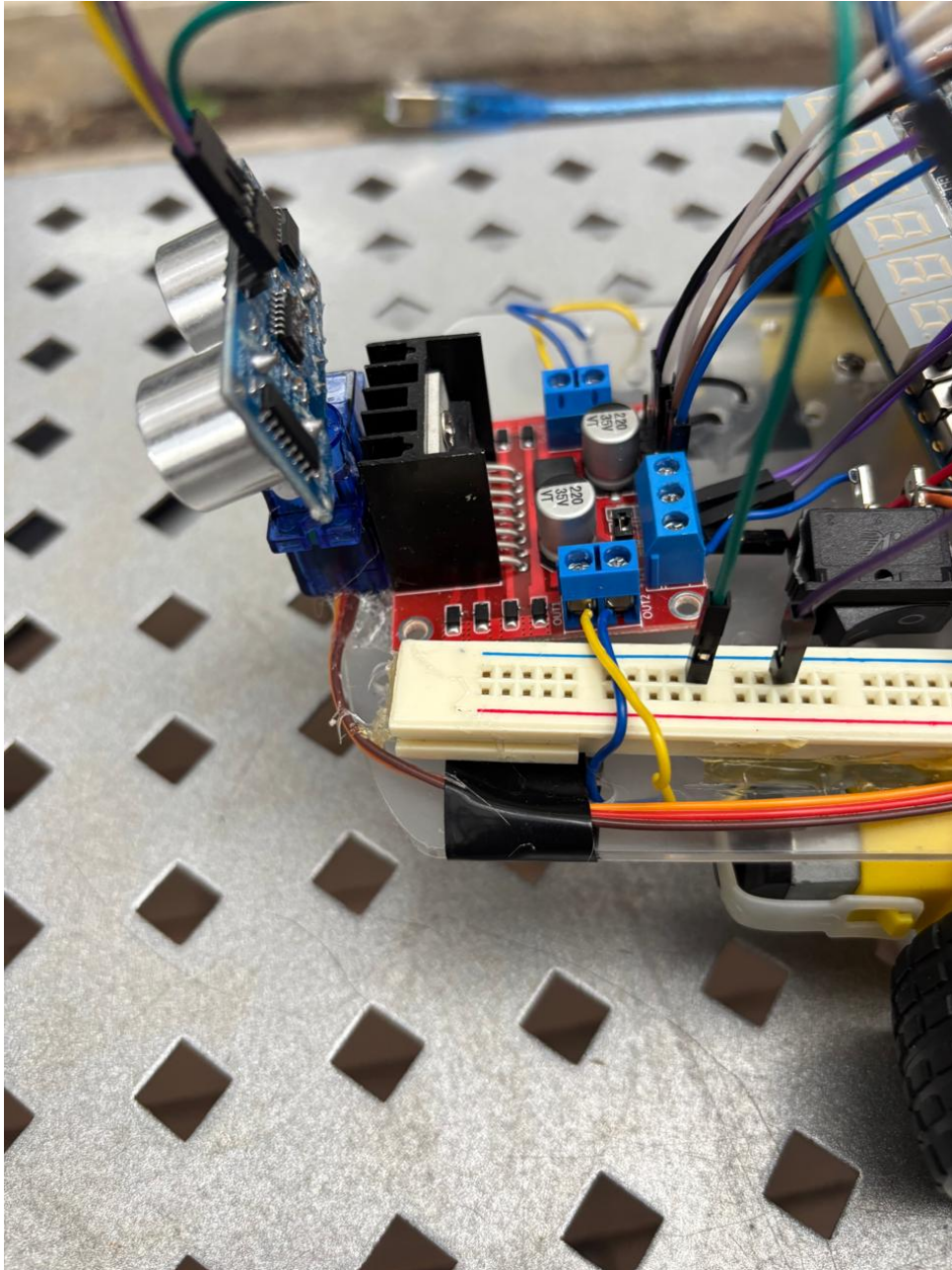


Figura 10: Foto del frente del robot

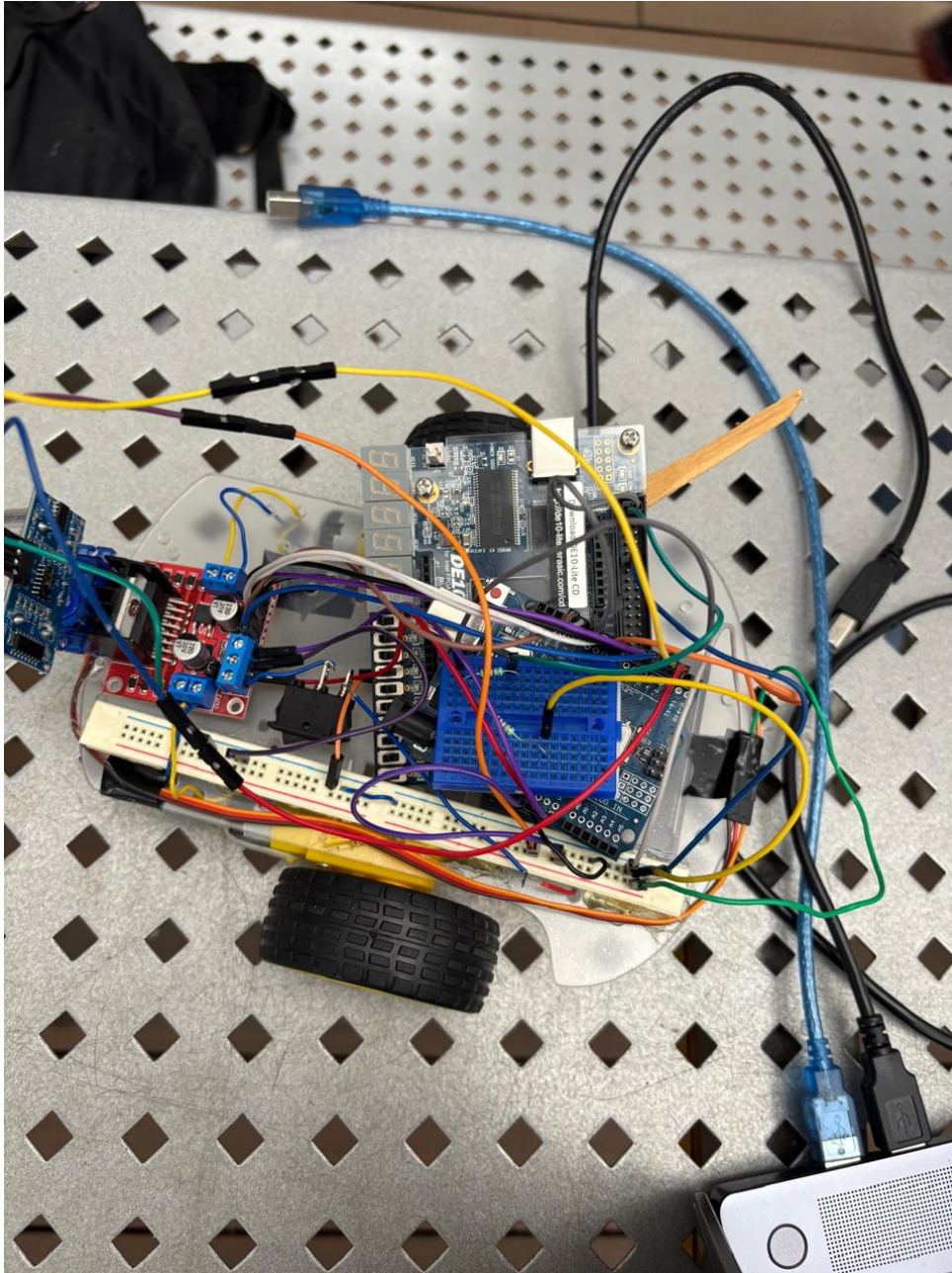


Figura 11: Foto de las conexiones

C. Datasheets de referencia

- HC-SR04 – Sensor ultrasónico (datasheet adjunto)
- L298N – Driver de motores puente H dual (datasheet adjunto)
- DE10-Lite – Manual de usuario FPGA: https://fpgacademy.org/Downloads/DE10_Lite_User_Manual.pdf
- ATmega328P – Microcontrolador Arduino UNO (datasheet adjunto)

D. Enlace al repositorio

https://github.com/Davidmunmen/proyecto_robot

The screenshot displays the GitHub interface for the repository 'proyecto_robot' by user 'Davidmunmen'. The repository is public and has 2 branches and 0 tags. A merge pull request #1 from 'Davidmunmen/doc/archivos-proyecto' is shown, merged by 'Carloosvii' yesterday with 5 commits. The file list includes:

- mini_carrito/codigo: detección de proximidad y top (2 months ago)
- README.md: Update README (3 weeks ago)
- arduino_con_fpga.ino: Agregue documentos que usamos para el proyecto del robot (yesterday)
- robot_principiante_v2.ino: Agregue documentos que usamos para el proyecto del robot (yesterday)
- seven_seg_display.v: Agregue documentos que usamos para el proyecto del robot (yesterday)
- top_module (1).v: Agregue documentos que usamos para el proyecto del robot (yesterday)
- top_module (2).v: Agregue documentos que usamos para el proyecto del robot (yesterday)
- uart_rx.v: Agregue documentos que usamos para el proyecto del robot (yesterday)
- uart_tx.v: Agregue documentos que usamos para el proyecto del robot (yesterday)

The README section is titled 'Proyecto Final: Diseño Digital VLSI' and describes the project as a self-driving robot system based on a DE10-Lite FPGA. It mentions the use of a Finite State Machine (FSM) implemented in VHDL/Verilog for logic decisions and communication with an Arduino acting as a bridge for sensor and motor control.

On the right sidebar, the 'About' section identifies the project as 'Project of VLSI'. The 'Releases' section shows no published releases. The 'Packages' section shows no published packages. The 'Contributors' section lists 'Davidmunmen David' and 'Carloosvii Carlos López Estrada'. The 'Languages' section shows a bar chart with the following data:

Language	Percentage
Verilog	50.7%
C++	32.4%
VHDL	16.9%

Figura 12: Repositorio GitHub